

Groupe B2-3 : Ivann Benini, Nicolas Lenois-Gelas, Jordy Rahantanirina, Noura Isaa

Document Workshop2026-B2g3-dossier

Sommaire

1. Présentation du projet....3
    1.1 Le point de départ et la mission....3
    1.2 Pourquoi ce projet mérite d'exister dans le RickLab....3
    1.3 Les fonctions de l'outil....3
2. Processus de conception....4
    2.1 Recherches....4
    2.2 Idées abandonnées et évolutions du projet....4
    2.3 Choix techniques....4
    2.4 Organisation du travail....5
3. Documentation technique....5
    3.1 Architecture du projet....5
    3.2 Composants utilisés....5
    3.3 Structure logicielle et explication du code....6
    3.4 Bibliothèques et API....7
    3.5 Intelligence artificielle utilisée....7
4. Fonctionnement détaillé....7
    4.1 Séquence d'initialisation....7
    4.2 Fonction télémètre....7
    4.3 Le langage de clics....7
    4.4 Fonction pointeur à charge....8
    4.5 Fonction rafale....8
    4.6 Gestion de la charge et recharge....8
    4.7 Analyseur de style....8
    4.8 Veille et réveil....8
    4.9 Les cibles réactives....9
5. Fabrication....9
    5.1 Machines et outils du myDiL....9
    5.2 Pièces imprimées....9
    5.3 Prototypage et assemblage....10
6. Difficultés et limites....11
    6.1 Ce qui n'a pas fonctionné....11
    6.2 Limites actuelles du prototype....12
    6.3 Ce que nous aurions amélioré....12
    6.4 Comment nous aurions poursuivi avec plus de temps....12
7. Gestion de projet....13
    7.1 Répartition des rôles....13
    7.2 Suivi des tâches et outils....13
    7.3 Déroulé de la semaine....13
    7.4 Ce que nous avons appris....14
Annexe A. Programme du Multitool-M4....16
Annexe B. Programme des cibles....26
Annexe C. Schéma de branchement des composants....29

1. Présentation du projet

1.1 Le point de départ et la mission

Tout commence par un portail qui s'ouvre dans la salle. Rick en sort, pose son pistolet laser sur la table et nous explique qu'il est cassé. Il n'a ni le temps ni l'envie de s'en occuper, alors il nous confie une mission : le réparer, et surtout l'améliorer avant de le lui rendre.

Réparer ne suffisait pas. Un pistolet qui ne fait qu'envoyer un faisceau reste un objet à une seule fonction, et ça n'intéresse pas Rick. Nous avons donc reconstruit l'objet de zéro pour en faire un outil complet.

Le Multitool-M4 est le résultat de cette mission. C'est un seul boîtier, tenu d'une main, qui regroupe plusieurs outils : un télémètre à ultrasons, un pointeur laser à charge progressive, un mode rafale, un compteur de charges avec recharge, un analyseur de style et une mise en veille automatique. Tout se commande avec un seul bouton. C'est la durée de l'appui et le nombre de clics qui décident de l'outil utilisé.

Restait un point à régler pour prouver que l'arme réparée fonctionne. Un pointeur qui envoie un faisceau ne dit pas s'il a touché quelque chose. Nous avons donc fabriqué deux cibles capables de répondre, pour pouvoir tester le tir et le montrer à Rick. Ce sont des extraterrestres imprimés en 3D qui passent du vert au rouge quand ils sont touchés.

1.2 Pourquoi ce projet mérite d'exister dans le RickLab

Le RickLab attend des objets qui interagissent vraiment avec leur environnement, et Rick attend mieux que son arme d'origine. Le Multitool-M4 mesure ce qui l'entoure, affiche le résultat, réagit à la façon dont on appuie sur son bouton, envoie un signal physique, et un autre objet lui répond. La boucle est complète et surtout elle est visible : on mesure, on charge, on tire, la cible change de couleur et se met à crier.

C'est aussi un objet volontairement excessif, et c'est voulu : on ne rend pas à Rick un simple pistolet réparé. Personne n'avait besoin de regrouper six outils dans un boîtier commandé par un seul bouton. C'est justement ce qui rend le projet intéressant. Il a fallu inventer un langage de clics, gérer une dizaine d'états sans jamais bloquer le programme, et faire tenir tout ça sur une seule petite carte.

> *« La frontière entre le génie et une très mauvaise idée, c'est juste la qualité du prototype. »* Rick Sanchez

1.3 Les fonctions de l'outil

Le tableau ci-dessous résume ce que l'objet sait faire. Chaque ligne correspond à une fonction réellement présente dans le code fourni en annexe.

Fonction	Ce qu'elle fait	Comment on l'active
Télémètre	Mesure la distance en continu et l'affiche en centimètres sur l'écran	Automatique, dès que l'outil est au repos
Pointeur à charge	Charge pendant 3 secondes avec montée lumineuse et sonore, puis envoie le faisceau	Appui maintenu sur le bouton
Mode rafale	Envoie des impulsions répétées de 90 ms toutes les 240 ms	1 clic court puis appui maintenu
Analyseur de style	Rend un verdict affiché à l'écran, avec carillon de validation ou de refus	1 clic court, ou 2 clics courts
Gestion de charge	Compte 20 charges, signale le vide et recharge en 2 secondes	3 clics courts pour recharger
Veille	Éteint l'écran et les LED pour économiser l'énergie	4 clics courts, ou 2 minutes sans action
Cibles réactives	Deux extraterrestres qui passent du vert au rouge quand ils sont touchés	Viser le ventre d'une cible et tirer

2. Processus de conception

2.1 Recherches

Les recherches se sont appuyées sur quatre ressources.

- Les documents fournis par les enseignants, qui servent de base commune à tous les groupes et qui cadrent ce qui est attendu pendant la semaine.
- L'assistant IA Claude, utilisé pour obtenir des explications, débloquer des points de code et produire les supports du projet.
- Le catalogue de composants disponibles au myDiL, pour savoir ce qu'il était réaliste de monter en cinq jours.
- La documentation des modules retenus : le fonctionnement du capteur HC-SR04, le pilotage d'un afficheur LCD par bus I2C, et la manière dont un récepteur laser signale qu'il reçoit un faisceau.

2.2 Idées abandonnées et évolutions du projet

Point de départ	Ce que c'est devenu, et pourquoi
Réparer le pistolet de Rick à l'identique	Abandonné. Un objet qui ne sait qu'envoyer un faisceau reste une fonction unique et ne mène nulle part techniquement. La mission est devenue une amélioration complète, et le pistolet est devenu un outil multifonction, ce qui donne bien plus à montrer devant un jury
Les noms « Le Portalyseur » puis « L'Interdimensionneur »	Remplacés par Multitool-M4. Les deux premiers noms promettaient une fonction que l'objet ne remplit pas. Le nom actuel dit simplement ce que fait l'outil
Le premier module laser récupéré	Son faisceau était trop faible. Il se voyait à peine sur les cibles et le résultat d'un tir n'était pas lisible pendant une démonstration. Nous l'avons remplacé par un module plus puissant, assez visible pour que le jury suive le tir, tout en restant un simple pointeur que l'on ne dirige jamais vers une personne
Envoyer le faisceau dans le vide et croire sur parole que ça marche	Remplacé par deux cibles réactives. C'est l'évolution la plus importante du projet : elle rend le tir vérifiable devant un jury
Un bouton par fonction	Impossible à loger dans la coque. Remplacé par un langage de clics sur un bouton unique, ce qui a demandé beaucoup plus de travail côté logiciel mais beaucoup moins côté mécanique

> *« Les meilleures inventions commencent souvent par une idée qui semblait mauvaise au départ. »* Rick Sanchez

2.3 Choix techniques

Chaque choix ci-dessous a été fait pendant la semaine et se retrouve directement dans le code fourni en annexe.

Choix	Justification
Carte NodeMCU ESP8266 dès le départ	C'est la carte que nous avons prise au myDiL et nous n'en avons pas changé. Elle est compacte, alimentée et programmée par un simple câble micro-USB, et elle offre assez de sorties pour tout piloter. Le Wi-Fi intégré était un plus, mais il n'a finalement pas servi
Deux cartes indépendantes plutôt qu'une seule	Les cibles doivent pouvoir être posées à plusieurs mètres de l'utilisateur. Tirer des fils entre l'outil et les cibles aurait supprimé tout l'intérêt de la démonstration. Chaque ensemble a donc sa carte et son alimentation, et le seul lien est le faisceau laser
Un seul bouton pour toutes les fonctions	La coque est petite et percée au plus juste. Plutôt que d'ajouter des boutons, les fonctions sont encodées dans la durée de l'appui et le nombre de clics. C'est ce qui fait du M4 un multitool et non un simple pointeur
Machine à états non bloquante plutôt qu'une suite de delay()	Pendant la charge, l'outil doit continuer à mesurer, à afficher et à surveiller le bouton. Tout repose donc sur millis() et sur une énumération d'états, jamais sur des pauses bloquantes

Choix	Justification
Armement préalable des cibles	Un personnage ne peut être touché que si son capteur a d'abord été vu au repos. Sans cette condition, une cible déjà éclairée au démarrage serait déclarée morte immédiatement

2.4 Organisation du travail

Nous nous sommes réparti les tâches de façon globale, sans rôles fixes, puis chacun est passé sur la partie où il était le plus à l'aise pour y apporter son expertise. Tout le monde a donc touché un peu à tout : électronique, code, modélisation et supports, tout en donnant un coup de main plus poussé là où il était le plus utile.

Ce fonctionnement a permis à chacun de comprendre l'ensemble du prototype, ce qui est utile au moment des questions du jury. Il demande en revanche de se parler souvent, puisque personne n'est propriétaire d'une partie précise.

3. Documentation technique

3.1 Architecture du projet

Le système repose sur deux cartes NodeMCU ESP8266 indépendantes. La première pilote le Multitool-M4 : bouton, capteur de distance, écran, LED RGB, buzzer et émetteur laser. La seconde pilote les deux cibles : deux récepteurs laser, deux LED RGB et un buzzer partagé. Les deux cartes ne sont reliées par aucun fil et n'échangent aucune donnée. La seule information qui passe de l'une à l'autre est la lumière du laser.

Le détail des broches utilisées se trouve en tête de chaque programme, en annexe A et en annexe B, et le schéma de branchement complet est donné en annexe C, figure 7. Chaque affectation y est commentée avec le repère sérigraphié sur la carte.

3.2 Composants utilisés

Le Multitool-M4 :

Réf.	Composant	Qté	Caractéristiques	Rôle dans le projet
U1	Carte NodeMCU ESP8266	1	Carte de développement, logique 3,3 V	Pilote tout l'outil : capteur, bouton, machine à états, affichage, sons et laser
U2	Capteur de distance à ultrasons	1	HC-SR04, portée 2 à 400 cm	Fonction télémètre : mesure la distance de ce que vise l'outil
U3	Écran LCD 1602	1	16 colonnes sur 2 lignes, liaison I2C	Affiche la distance, la charge restante, les barres de progression et les verdicts
U4	Module laser	1	Module laser rouge de faible puissance	Fonction pointeur : produit le faisceau, aligné sur l'axe de visée
U5	Buzzer	1	Buzzer passif	Tous les retours sonores : démarrage, charge, tir, recharge et verdicts
D1	Module LED RGB	1	LED RGB pilotée en PWM	Indicateur d'état de l'outil. Seuls le rouge et le vert sont utilisés, leur mélange donne l'orange. Le canal bleu n'est pas utilisé
S1	Bouton poussoir	1	Contact simple, tirage interne	Commande unique de l'outil. Toutes les fonctions passent par lui
M1	Alimentation	1	USB 5 V	Alimente la carte et les périphériques

Réf.	Composant	Qté	Caractéristiques	Rôle dans le projet
M2	Coque imprimée en 3D	1	PLA, plusieurs pièces	Châssis, poignée, capot et logements des composants

Les deux cibles :

Réf.	Composant	Qté	Caractéristiques	Rôle dans le projet
U6	Carte NodeMCU ESP8266	1	Seconde carte, alimentée séparément	Lit les deux récepteurs et pilote les LED et le buzzer des cibles
U7, U8	Module récepteur laser	2	Photodétecteur sur platine, sortie numérique	Logé dans le ventre de chaque personnage. C'est la zone à toucher
U9, U10	Module LED RGB	2	LED RGB sur platine, canal bleu non utilisé	Verte quand le personnage est en vie, rouge quand il est touché
U11	Buzzer	1	Buzzer passif	Joue le son de KO, partagé par les deux cibles
C1, C2	Coque extraterrestre imprimée en 3D	2	PLA bleu	Corps de la cible. Ouverture au ventre et trou au sommet du crâne
P1	Panneau en carton	1	Plaque de carton rigide	Support des deux cibles, collées en façade. La carte et le câblage sont fixés au dos, hors de vue
M3	Câblage de liaison	2 lots	Nappes Dupont	Relient chaque cible à la carte qui la commande

3.3 Structure logicielle et explication du code

Les deux programmes sont écrits en C++ Arduino et fournis intégralement en annexe. Aucun des deux n'utilise de pause bloquante. Tout repose sur la fonction millis(), qui donne le temps écoulé depuis le démarrage. C'est ce qui permet à l'outil de mesurer, d'afficher et d'écouter le bouton en même temps qu'il joue un son de charge.

Le programme du Multitool-M4

- Une énumération EtatTir définit dix états : REPOS, CHARGEMENT, PAUSE_TIR, TIR, TIR_AUTO, ATTENTE_CLIC, RECHARGE, CONFIRMATION, VIDE et VEILLE. À chaque instant, l'outil est dans un seul de ces états. C'est cette machine à états qui permet à un bouton unique de commander autant de fonctions.
- La fonction actualiserCharge() est le cœur du programme. Elle lit le bouton, gère l'anti rebond, compte les clics, fait avancer l'état courant et pilote les LED, le laser et le buzzer.
- Cette fonction est appelée deux fois dans loop(), et aussi à l'intérieur de mesurerDistance(). C'est volontaire : la lecture de l'écho prend un peu de temps, pendant lequel le son de charge se figerait si rien ne tournait.
- La fonction loop() ne décide presque rien. Elle se contente de choisir quel écran afficher en fonction de l'état courant.
- L'affichage est protégé par un indicateur, feuAffiche, et par la comparaison du pourcentage précédent. Un texte n'est réécrit sur l'écran que s'il a changé, sinon l'affichage scintille.

Le programme des cibles

- Il est beaucoup plus court et construit en deux blocs symétriques, un par personnage.
- Chaque personnage a trois variables : mort, pret et la date de l'impact. La variable pret impose que le capteur ait d'abord été vu au repos avant qu'un tir puisse compter.
- Le son de KO est joué note par note par mettreAJourSon(), sans jamais bloquer la boucle. Une fréquence de zéro dans le tableau correspond à une courte pause.
- Le retour au vert se fait cinq secondes après l'impact, par simple comparaison de dates.

3.4 Bibliothèques et API

Élément	Usage
Arduino.h	Fonctions de base de la plateforme Arduino
Wire.h	Bus I2C, initialisé sur les broches D2 et D1 pour l'écran
LiquidCrystal_I2C.h	Pilotage de l'afficheur LCD 1602 à l'adresse 0x27

Le projet n'utilise aucune API externe, aucun serveur MQTT et aucun service en ligne. Le Wi-Fi de l'ESP8266 n'est pas activé. Tout se passe en local, ce qui rend la démonstration indépendante du réseau de l'école.

3.5 Intelligence artificielle utilisée

Le sujet autorise l'usage d'assistants IA à condition de rester maître du projet. Nous avons utilisé l'assistant Claude, en complément des documents fournis par les enseignants.

Claude nous a servi à trois choses : débloquer des points de code quand la liaison entre les périphériques ne fonctionnait pas, obtenir des explications sur le fonctionnement de certains modules, et produire les supports de communication du projet, à savoir l'affiche, la présentation et ce dossier.

Notre présentation cite d'ailleurs l'IA parmi les moyens qui nous ont permis de débloquer la situation, aux côtés de la documentation et de l'aide extérieure.

Le code présent en annexe reste sous notre responsabilité. Chaque état de la machine à états, chaque broche et chaque temporisation peuvent être expliqués, modifiés et corrigés par les membres du groupe.

4. Fonctionnement détaillé

4.1 Séquence d'initialisation

Dès la mise sous tension, l'outil joue une animation d'initialisation de deux secondes. L'écran affiche INITIALISATION avec une barre de progression. La LED passe au vert en montant en intensité et le buzzer émet trois bips courts. L'écran affiche ensuite SYSTEME PRET et la charge disponible.

4.2 Fonction télémètre

Au repos, le capteur mesure la distance en permanence. La carte envoie une courte impulsion sur TRIG. Le capteur renvoie sur ECHO une impulsion dont la durée correspond à l'aller-retour de l'onde. La distance se déduit de cette durée et s'affiche en centimètres.

La lecture est protégée par un délai maximal. Si rien ne revient, l'écran affiche AUCUNE MESURE. L'affichage se rafraîchit plusieurs fois par seconde. La première ligne montre la distance et la charge restante, la seconde la valeur en centimètres.

4.3 Le langage de clics

C'est le cœur du multitool. Tout se pilote avec un seul bouton et c'est la durée de l'appui et le nombre de clics qui décident de la fonction. Un appui bref compte comme un clic court, deux clics rapprochés sont regroupés, et un filtre anti rebond écarte les faux contacts.

Geste sur le bouton	Fonction déclenchée
Appui maintenu plus de 0,5 s	Pointeur à charge : 3 secondes de charge puis tir
1 clic court puis appui maintenu	Mode rafale

Geste sur le bouton	Fonction déclenchée
1 clic court seul	Analyseur de style, verdict positif
2 clics courts	Analyseur de style, verdict négatif
3 clics courts	Recharge du compteur
4 clics courts	Mise en veille immédiate
Appui de 2 secondes pendant la veille	Réveil de l'outil

L'écran accompagne l'utilisateur pendant la saisie. Il affiche le nombre de clics déjà enregistrés et rappelle la fonction que le prochain geste va déclencher. Personne n'a donc besoin de connaître la liste par cœur, ce qui compte beaucoup pendant une démonstration de cinq minutes.

4.4 Fonction pointeur à charge

Quand la charge démarre, la LED passe progressivement du vert au rouge. En même temps elle clignote de plus en plus vite et le son monte, avec une montée qui accélère vers la fin. L'écran affiche CHARGEMENT et une barre de progression.

La charge dure trois secondes. À la fin, la LED devient rouge fixe, le système marque une brève pause, puis le faisceau part. Le laser s'allume et le buzzer lâche un son descendant. Le laser reste allumé tant que le bouton n'est pas relâché, ce qui laisse le temps d'ajuster la visée sur le ventre d'une cible. Chaque tir consomme une charge.

4.5 Fonction rafale

Un clic court suivi d'un appui maintenu déclenche le mode rafale. Le laser s'allume par impulsions courtes, séparées par un temps mort, ce qui permet aux récepteurs des cibles de distinguer chaque tir. Chaque cycle consomme une charge, et la rafale s'arrête dès que le compteur est vide.

4.6 Gestion de la charge et recharge

Le compteur contient 20 charges. Quand il est vide, la LED passe à l'orange et l'écran affiche CHARGEUR VIDE en rappelant le geste pour recharger. Trois clics courts lancent la recharge, qui dure quelques secondes. Pendant ce temps la LED revient de l'orange vers le vert et le buzzer émet des bips de plus en plus aigus. Un dernier bip confirme l'opération.

4.7 Analyseur de style

C'est la fonction la plus absurde de l'outil, et elle est assumée comme telle. Un clic court isolé rend un verdict positif, affiché à l'écran avec un message tiré au hasard parmi quatre et un carillon de quatre notes montantes. Deux clics courts rendent un verdict négatif, avec un message parmi trois et un carillon descendant. Le verdict reste trois secondes à l'écran.

Techniquement, cette fonction repose sur un tirage aléatoire initialisé par micros(), et sur deux tableaux de notes joués sans bloquer la boucle principale. Elle ne sert à rien, mais elle montre que la machine à états sait gérer une fonction qui n'a aucun rapport avec le laser.

4.8 Veille et réveil

Quatre clics courts mettent l'outil en veille. L'écran, le rétroéclairage et les LED s'éteignent. La veille se déclenche aussi toute seule après deux minutes sans action, ce qui évite d'oublier l'outil allumé entre deux démonstrations. Un appui de deux secondes sur le bouton le réveille.

4.9 Les cibles réactives

Chaque cible possède un récepteur laser dans le ventre et une LED RGB dans la tête. Le buzzer est partagé par les deux personnages.

Le récepteur passe à l'état bas quand il reçoit la lumière du laser. Quand le faisceau atteint le ventre, la LED de la tête passe au rouge et le buzzer joue un son de KO, une suite de notes descendantes dans l'esprit des bornes d'arcade. Cinq secondes plus tard, le personnage revient automatiquement au vert.

Les deux capteurs sont lus séparément. Le système sait donc laquelle des deux cibles a été touchée. La figure 1 montre les deux personnages et le montage qui les commande.

Les deux personnages sont collés sur un panneau en carton. La carte, le buzzer et tout le câblage sont fixés au dos du panneau, donc invisibles de face. Vu du tireur, il ne reste que les deux extraterrestres, ce qui rend la démonstration beaucoup plus lisible.

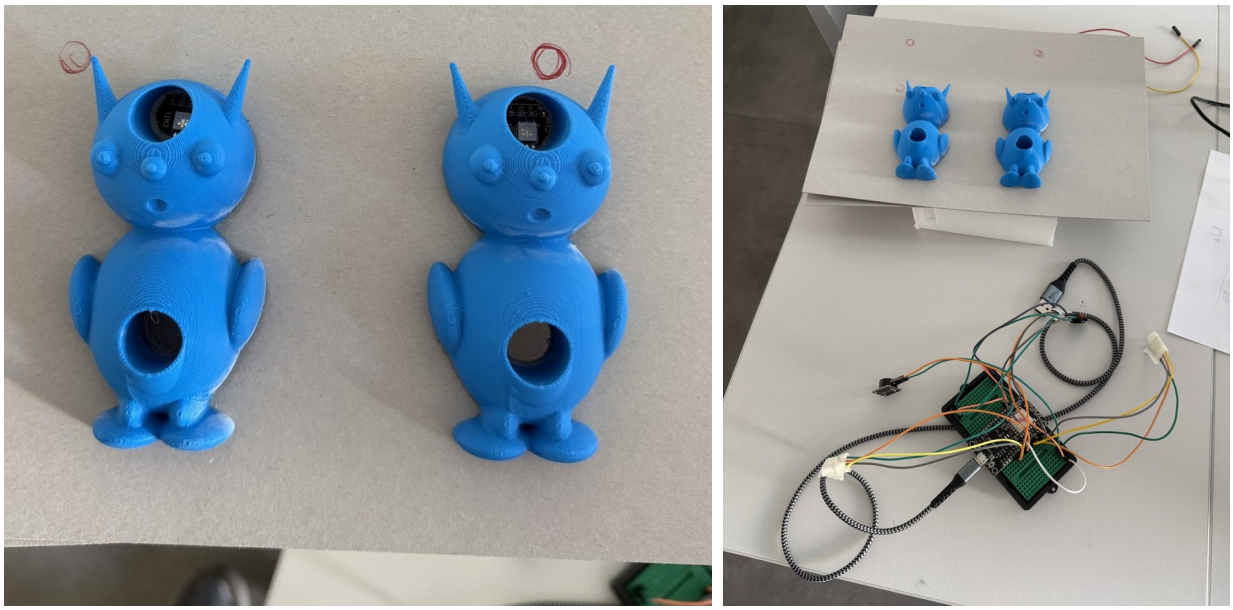


Figure 1 : les deux cibles imprimées en 3D, avec le récepteur au ventre et le module LED au sommet du crâne. À droite, le montage qui les commande.

5. Fabrication

5.1 Machines et outils du myDiL

Le projet utilise l'imprimante 3D du myDiL, pilotée depuis BambuLab Studio. Toutes les pièces du prototype sont imprimées en PLA.

5.2 Pièces imprimées

Pièce	Quantité	Description
Coque du Multitool-M4	1 en 2 parties	Corps rectangulaire et poignée inclinée, avec logements pour l'écran et les modules. Partie basse puis partie haute
Coque extraterrestre	2	Personnage debout, ouverture circulaire au ventre et trou au sommet du crâne

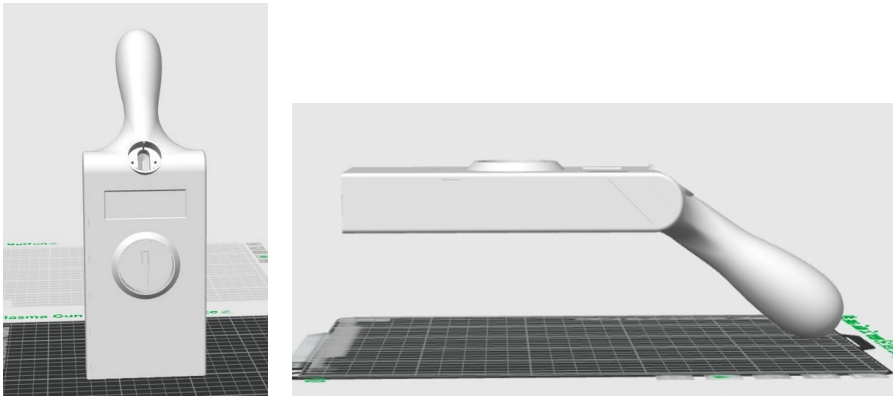


Figure 2 : modèle 3D de la coque du Multitool-M4, vue de face et vue de profil.

Le modèle 3D de la coque est reproduit en figure 2. Les repères suivants y sont visibles.

- Logement rectangulaire : encastrement de l'écran.
- Logement circulaire principal : réception d'un module à travers la coque.
- Logement circulaire supérieur, à la jonction de la poignée : passage de câblage et fixation.
- Encombrement : un boîtier qui tient dans une main, de la taille d'un gros pistolet jouet.

5.3 Prototypage et assemblage

Le montage a d'abord été validé entièrement sur platine d'essai, composant par composant, avant toute intégration dans les coques, comme le montre la figure 3. Cette étape a permis de vérifier le capteur, l'écran, le laser et les sons séparément, puis ensemble.

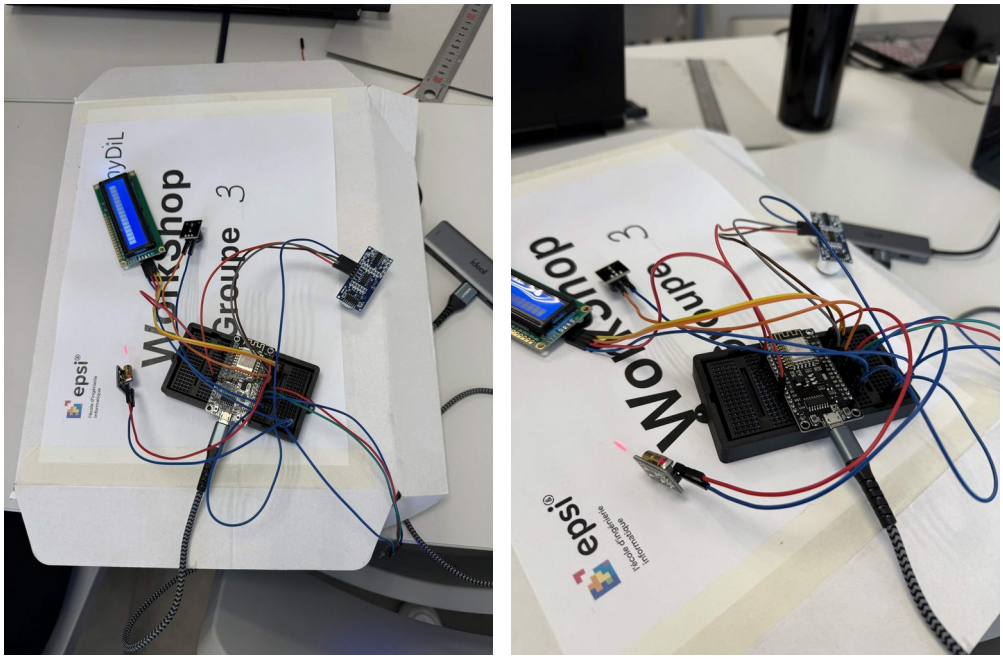


Figure 3 : montage de validation du Multitool-M4 sur platine d'essai, avant intégration.

Pour les cibles, l'assemblage consiste à insérer le récepteur dans l'ouverture ventrale et la platine LED en force dans le trou du crâne, puis à faire ressortir la nappe par l'arrière du personnage.

État d'assemblage

Le Multitool-M4 est aujourd'hui monté dans sa coque imprimée, visible sur la figure 4. La carte et le câblage sont maintenus à l'intérieur du corps, la poignée est solidaire, et les modules sont installés dans leurs logements. Au moment des photos, il ne restait plus qu'à refermer le capot supérieur.

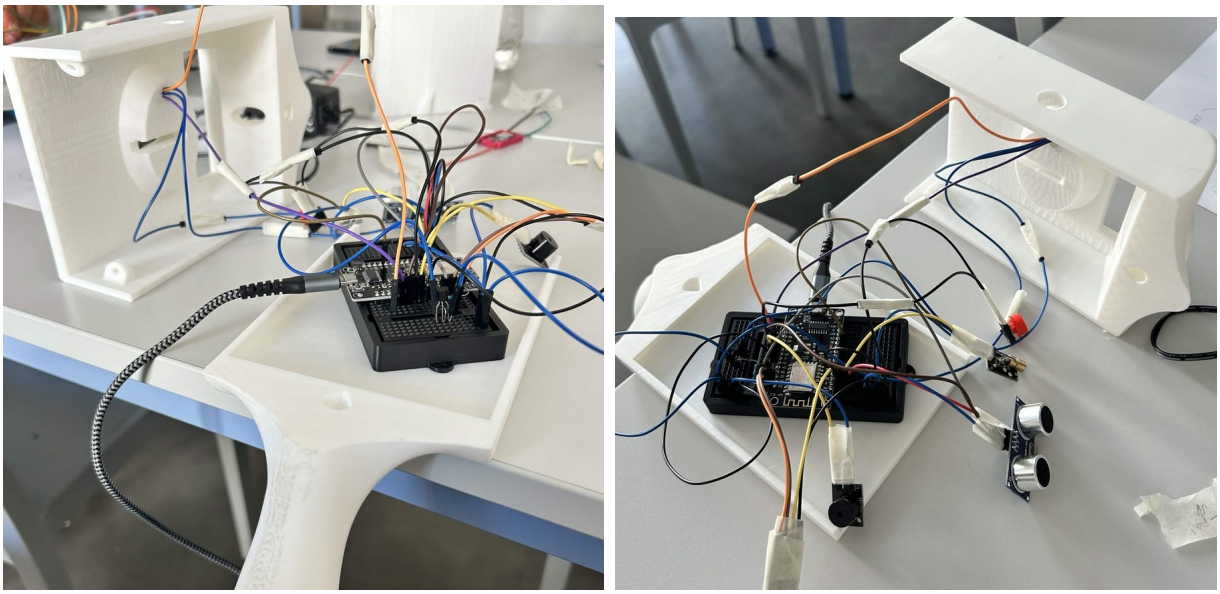


Figure 4 : le Multitool-M4 monté dans sa coque, avant fermeture du capot. À droite, les modules installés dans le corps.

Les deux cibles sont également terminées. Les extraterrestres sont collés sur la face avant du panneau en carton et l'ensemble de l'électronique est reporté au dos, hors de vue, comme sur la figure 5.

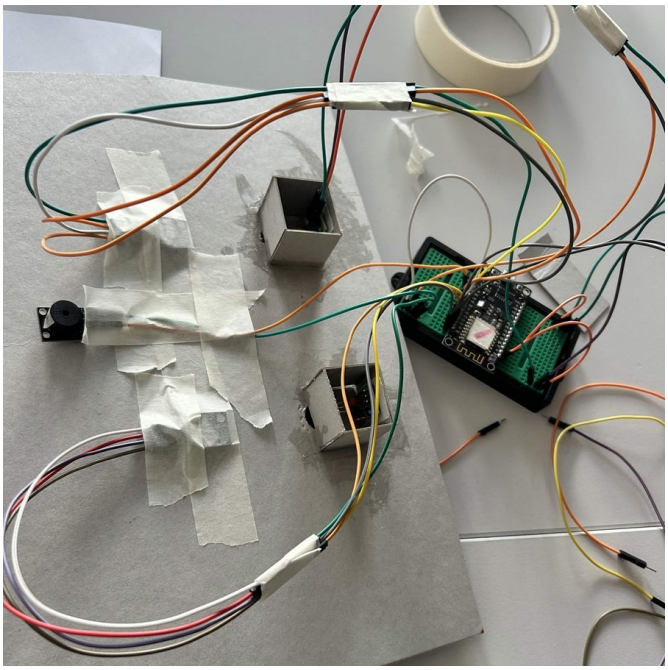


Figure 5 : le dos du panneau des cibles, avec la seconde carte, le buzzer et le câblage.

6. Difficultés et limites

6.1 Ce qui n'a pas fonctionné

La liaison entre le logiciel et le matériel a été notre principal point de blocage. Faire fonctionner ensemble tous les périphériques sur une même carte a demandé nettement plus de temps que prévu. Pris séparément, chaque module marchait. Réunis, ils se gênaient.

Trois causes reviennent :

- Des erreurs de branchement, notamment entre TX et RX, que nous avons failli inverser deux fois. La LED s'est mise à clignoter sans raison apparente avant que nous comprenions d'où venait le problème. C'est pour cette raison que

le canal rouge de la LED est aujourd'hui câblé sur la broche TX, ce qui interdit d'utiliser la liaison série pour déboguer l'outil.

- Un code devenu trop complexe. Chaque fonction ajoutée au multitool ajoute des états et des gestes de bouton, et la fonction principale est devenue longue et difficile à relire.
- Trop de sollicitations dans la boucle. Il a fallu espacer les rafraîchissements de l'écran et les mesures pour que l'ensemble reste fluide.

Nous avons aussi cramé un récepteur laser pendant les essais. Une erreur de branchement a suffi à le mettre hors service, et il a fallu le remplacer puis reprendre les tests de cette partie. On ne fait pas d'omelette sans casser des œufs : l'équipe a rebondi le jour même, en repartant du montage propre et en vérifiant chaque liaison avant de rebrancher.

Ces blocages ont été levés en combinant trois ressources : les documents fournis par les enseignants, l'aide extérieure et l'assistant Claude.

6.2 Limites actuelles du prototype

- Le capteur à ultrasons perd la mesure sur les surfaces molles et sur les angles. L'écran affiche alors AUCUNE MESURE.
- Le récepteur des cibles est sensible à la lumière ambiante. Un éclairage direct dans l'axe peut provoquer de fausses détections.
- Le canal bleu n'est utilisé nulle part, ni sur la LED de l'outil ni sur celles des cibles. Les deux couleurs suffisent pour le dégradé du vert au rouge.
- Il n'y a pas de comptage de points. Le système sait qui est touché, mais ne garde aucun score.
- Le câblage reste sur platine d'essai à l'intérieur de la coque. Il n'y a ni circuit imprimé ni soudure, donc un fil peut se déplacer si l'objet est manipulé sans précaution.

6.3 Ce que nous aurions amélioré

- Câbler plus proprement dès le départ et vérifier chaque liaison avant d'écrire la moindre ligne de code. C'est le point que nous identifions nous-mêmes comme le plus coûteux en temps.
- Prendre le temps d'une compréhension globale du montage par tout le monde, plutôt que d'avancer en parallèle sur des morceaux.
- Découper le programme en plusieurs fichiers, un par fonction du multitool, pour qu'il reste relisible.

6.4 Comment nous aurions poursuivi avec plus de temps

- Ajouter un comptage de points et un vrai mode de jeu, avec un nombre de cibles à atteindre dans un temps donné.
- Faire communiquer les deux cartes sans fil, par exemple en ESP-NOW, pour centraliser le score sur l'écran de l'outil. Le Wi-Fi de l'ESP8266 est déjà présent et inutilisé.
- Ajouter d'autres outils au multitool, par exemple un niveau à bulle numérique ou un thermomètre, puisque la machine à états accepte de nouvelles fonctions sans refonte.
- Remplacer la platine d'essai par un circuit soudé et passer sur une batterie embarquée, pour que l'outil soit enfin indépendant d'un câble USB.
- Augmenter le nombre de cibles, la structure du programme le permet sans changement profond.

7. Gestion de projet

7.1 Répartition des rôles

La répartition est décrite au paragraphe 2.4 : des tâches attribuées de façon globale, sans rôles fixes, puis chacun sur la partie où il apporte le plus. Personne n'est donc propriétaire d'un morceau du prototype, et chacun peut répondre sur l'ensemble devant le jury.

7.2 Suivi des tâches et outils

Outil	Usage
Trello	Suivi des tâches et répartition du travail au jour le jour
GitHub	Versionnement du code source des deux cartes. Dépôt du projet : https://github.com/JordyEPSI/WS-Rick-Morty-B2.git
Arduino IDE	Développement et téléversement des programmes
BambuLab Studio	Préparation des impressions 3D
Imprimante 3D du myDiL	Fabrication de la coque et des deux cibles
Claude	Assistant IA pour le code, les explications et les supports du projet

Le tableau Trello du groupe est organisé en une liste par jour. Chaque carte correspond à une tâche réellement traitée pendant la semaine, et c'est ce tableau qui a servi de base au déroulé du paragraphe suivant.

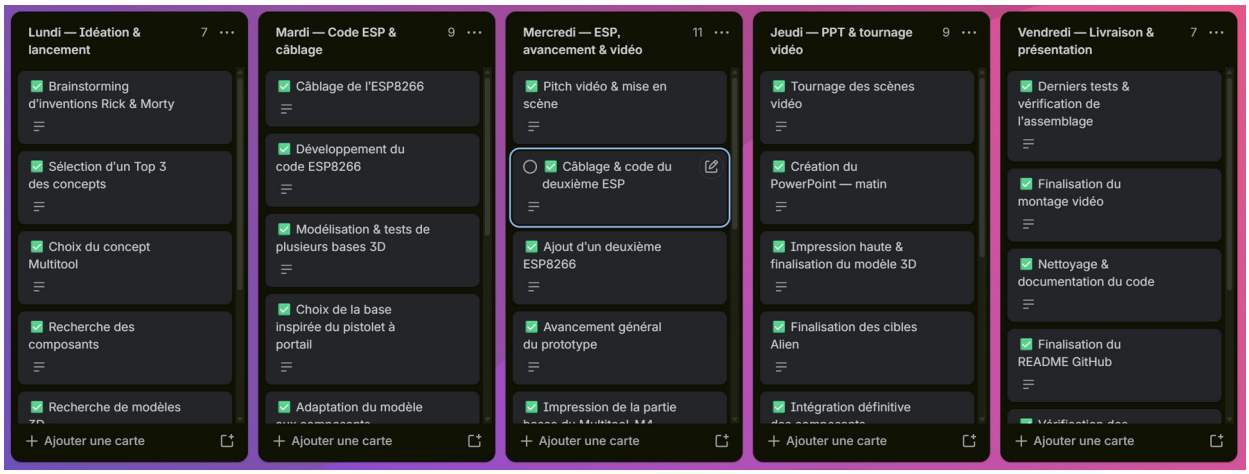


Figure 6 : le tableau Trello du groupe, une liste par jour de la semaine.

7.3 Déroulé de la semaine

Le déroulé ci-dessous correspond au travail réellement effectué, tel qu'il a été suivi sur notre tableau Trello.

Lundi : brainstorming et lancement du projet

Équipe principale : Ivann et Jordy.

- Brainstorming autour de plusieurs inventions inspirées de Rick et Morty, puis sélection d'un top 3 des concepts.
- Comparaison des idées et choix progressif du pistolet multifonction.
- Recherche des composants nécessaires au prototype.
- Recherche de modèles 3D pouvant servir de base, puis début de la modélisation et des premières modifications dans Bambu Studio et Blender.
- Répartition du travail entre les membres de l'équipe.

Résultat de la journée : le concept du Multitool est retenu et les premières bases du prototype sont définies.

Mardi : modélisation 3D et électronique

Équipe principale : Ivann, Jordy, Nicolas et Noura.

- Poursuite de la modélisation du Multitool, avec l'étude et le test de plusieurs modèles 3D.
- Choix définitif d'une base inspirée du pistolet à portail de Rick, puis modification du modèle pour intégrer les composants électroniques et préparer l'espace intérieur.
- Début des branchements sur l'ESP8266.
- Branchement et premiers tests des composants : laser, écran LCD, capteur à ultrasons, LED, buzzer et bouton.
- Câblage de la carte ESP8266 et écriture du programme du Multitool, en parallèle du travail sur le modèle 3D.

Résultat de la journée : le modèle 3D est choisi et les premiers éléments électroniques fonctionnent.

Mercredi : impression 3D, intégration et cibles

- Impression de la partie basse du Multitool-M4, vérification des dimensions et placement de l'ESP8266 à l'intérieur.
- Ajustement de l'emplacement des différents composants et finalisation du fonctionnement du laser.
- Avancement et finalisation du code principal du Multitool.
- Modélisation et finalisation des cibles extraterrestres, puis impression des deux cibles en fin de journée.
- Ajout d'un deuxième ESP8266 pour gérer les cibles, câblage de cette seconde carte, écriture de son programme et premiers tests de détection et de réaction.
- Écriture du pitch et travail sur la mise en scène de la vidéo de présentation.

Résultat de la journée : la première partie physique du Multitool est assemblée et le système de cibles fonctionne.

Jeudi : assemblage et finalisation

- Impression de la partie haute du Multitool-M4 et finalisation du modèle 3D.
- Finalisation des deux cibles extraterrestres.
- Placement définitif des composants électroniques dans le Multitool et dans les cibles, puis assemblage des différentes parties imprimées.
- Finalisation du code du Multitool et des cibles.
- Tests complets du prototype : télémètre, pointeur laser, charge progressive, mode rafale, compteur de charges, recharge, LED, sons, mise en veille et détection des impacts sur les cibles.
- Préparation du support de présentation le matin.
- Tournage des scènes de la vidéo, correction des derniers problèmes rencontrés et début du montage.

Résultat de la journée : le Multitool-M4 et les cibles sont finalisés et testés.

Vendredi : présentation et livraison

- Derniers tests du prototype et vérification de l'assemblage.
- Finalisation du montage vidéo.
- Nettoyage et documentation du code, finalisation du README sur GitHub.
- Vérification des modèles 3D et de l'ensemble des fichiers du projet.
- Préparation de la démonstration, puis présentation finale du Multitool-M4 et des cibles.

7.4 Ce que nous avons appris

- Technologies : pas de découverte totale, mais une bien meilleure prise en main des logiciels de modélisation 3D et de l'électronique connectée.
- Méthodes : le suivi par Trello, le brainstorming, l'écoute et la communication au sein du groupe.
- Outils : la chaîne d'impression 3D avec BambuLab, de la modélisation à la pièce finie.

- Organisation : travailler sans rôles fixes fait progresser tout le monde, mais demande de se parler beaucoup plus souvent.

Annexe A. Programme du Multitool-M4

Programme téléversé sur la carte NodeMCU de l'outil. Les commentaires de description ont été retirés pour alléger la lecture. Seuls quelques rappels utiles ont été conservés.

```
// Bibliotheques : base Arduino, bus I2C et pilotage de l'ecran LCD
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Correspondance entre les reperes serigraphies sur la carte et les GPIO
#define D1 5
#define D2 4
#define D3 0
#define D4 2
#define D5 14
#define D6 12
#define D7 13
#define D8 15
#define D0 16

// Affectation des broches aux composants du multitool
#define pinBTN D6
#define pinLED D4
#define pinLASER D0
#define pinBUZZER D5
#define pinLED_R 1  // TX / GPIO1 : ne pas utiliser Serial.
#define pinLED_V D8

#define TRIG D3
#define ECHO D7  // ECHO via un diviseur de tension 5 V vers 3,3 V.

// LED integree a la carte : etat bas = allumee
#define ALLUME_LED LOW
#define ETEIND_LED HIGH

// Module laser : etat haut = faisceau actif
#define ALLUME_LASER HIGH
#define ETEIND_LASER LOW

LiquidCrystal_I2C lcd(0x27, 16, 2);   // Ecran 16 colonnes sur 2 lignes, adresse I2C 0x27

// Lecture du bouton, anti rebond et dates de reference
bool systemeActive = false;
int derniereLecture = HIGH;
int etatBouton = HIGH;
unsigned long dernierChangement = 0;
unsigned long derniereMesure = 0;
const unsigned long DUREE_CHARGE = 3000;   // Duree de la charge avant le tir
unsigned long debutCharge = 0;
unsigned long dernierEffet = 0;
const unsigned long DUREE_PAUSE = 80;
const unsigned long DUREE_SON_TIR = 180;
const unsigned long DELAI_VEILLE_AUTO = 120000;   // Veille automatique apres 2 minutes sans action
unsigned long derniereActivite = 0;

// Plages de frequences des sons de charge et de tir
const unsigned int FREQUENCE_CHARGE_DEBUT = 350;
const unsigned int FREQUENCE_CHARGE_FIN = 1800;
const unsigned int FREQUENCE_TIR_DEBUT = 3000;
const unsigned int FREQUENCE_TIR_FIN = 180;
unsigned long debutPhase = 0;
unsigned long debutPulsation = 0;
bool feuAffiche = false;
const unsigned long DELAI_DOUBLE_CLIC = 650;   // Au dela, deux clics ne sont plus groupes
const unsigned long DUREE_CLIC_COURT = 500;   // Limite entre un clic court et un appui long
const unsigned long DUREE_TIR_AUTO = 90;
const unsigned long PERIODE_TIR_AUTO = 240;
const int CAPACITE = 20;   // Nombre de charges du compteur
const int VERT_ORANGE = 90;
const unsigned long DUREE_RECHARGE = 2000;   // Duree de la recharge
const unsigned long DUREE_CONFIRMATION = 800;
const unsigned long DUREE_BIP_PRET = 80;
const unsigned long DUREE_VERDICT = 3000;

// Carillons de l'analyseur de style : validation puis refus
const unsigned int NOTES_VALIDATION[] = {784, 988, 1319, 1568};
```

```
const unsigned long DUREES_VALIDATION[] = {80, 90, 120, 220};
const int NOMBRE_NOTES_VALIDATION = sizeof(NOTES_VALIDATION) / sizeof(NOTES_VALIDATION[0]);
const unsigned int NOTES_REFUS[] = {1568, 1047, 698, 392};
const unsigned long DUREES_REFUS[] = {80, 100, 140, 260};
const int NOMBRE_NOTES_REFUS = sizeof(NOTES_REFUS) / sizeof(NOTES_REFUS[0]);

// Messages tires au hasard pour chaque verdict
const char* const MESSAGES_DRIP[] = {
  "Sexyyy nana!", "Sapeee!", "Hiiii trop frais", "Popopo ca claque"
};
const char* const MESSAGES_PAS_DRIP[] = {
  "Peut mieux faire", "Change la paire", "Non."
};
int munitions = CAPACITE;
int clicsCourts = 0;
bool verdictAffiche = false;
bool verdictDrip = false;
unsigned long debutVerdict = 0;
int indexVerdict = 0;
bool sonVerdictActif = false;
int noteVerdict = 0;
unsigned long debutNoteVerdict = 0;
unsigned long dernierCycleAuto = 0;
bool tirCycleActif = false;
bool attendreRelachement = false;
unsigned long dernierRelachement = 0;

// Les dix etats possibles du multitool
enum EtatTir { REPOS, CHARGEMENT, PAUSE_TIR, TIR, TIR_AUTO, ATTENTE_CLIC, RECHARGE, CONFIRMATION, VIDE, VEILLE };
EtatTir etatTir = REPOS;
int dernierPourcentage = -1;
unsigned long dernierAffichageCharge = 0;
const unsigned long DUREE_DEMARRAGE = 2000;
const unsigned long DUREE_REVEIL = 2000;
const unsigned long ANTI_REBOND = 30;
unsigned long debutAppui = 0;

// Animation de demarrage : barre de progression, LED verte et trois bips
void animerDemarrage() {
  digitalWrite(pinLASER, ETEIND_LASER);
  analogWrite(pinLED_R, 0);
  analogWrite(pinLED_V, 0);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("INITIALISATION  ");

  unsigned long debut = millis();
  int dernierProgres = -1;
  int dernierSon = -1;
  while (millis() - debut < DUREE_DEMARRAGE) {
    unsigned long ecoule = millis() - debut;
    int progres = ecoule * 100UL / DUREE_DEMARRAGE;
    analogWrite(pinLED_V, ecoule * 255UL / DUREE_DEMARRAGE);

    int son = ecoule < 60 ? 900 :
              (ecoule >= 700 && ecoule < 760 ? 1200 :
              (ecoule >= 1400 && ecoule < 1460 ? 1600 : 0));
    if (son != dernierSon) {
      if (son > 0) tone(pinBUZZER, son);
      else noTone(pinBUZZER);
      dernierSon = son;
    }

    if (progres / 5 != dernierProgres / 5 || dernierProgres < 0) {
      dernierProgres = progres;
      lcd.setCursor(0, 1);
      lcd.print("[");
      for (int i = 0; i < 10; i++) {
        lcd.print(i < progres / 10 ? "#" : " ");
      }
      lcd.print("]");
      if (progres < 100) lcd.print(" ");
      if (progres < 10) lcd.print(" ");
      lcd.print(progres);
      lcd.print("%");
    }
    delay(5);
  }
```

```
  noTone(pinBUZZER);
  analogWrite(pinLED_V, 255);
  attendreRelachement = true;
}

// Configuration des broches, du bus I2C et de l'ecran
void setup() {

  pinMode(pinBTN, INPUT_PULLUP);

  pinMode(pinLED, OUTPUT);
  pinMode(pinLASER, OUTPUT);
  pinMode(pinBUZZER, OUTPUT);

  pinMode(pinLED_R, OUTPUT);
  pinMode(pinLED_V, OUTPUT);
  analogWriteRange(255);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  digitalWrite(TRIG, LOW);
  digitalWrite(pinBUZZER, LOW);

  digitalWrite(pinLED, ETEIND_LED);
  digitalWrite(pinLASER, ETEIND_LASER);
  digitalWrite(pinLED_R, LOW);
  digitalWrite(pinLED_V, LOW);

  Wire.begin(D2, D1);

  lcd.init();
  lcd.backlight();

  animerDemarrage();
  derniereActivite = millis();

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEME PRET");
  lcd.setCursor(0, 1);
  lcd.print("MUNITIONS 20/20");
}

// Coupe le laser, la LED temoin et le buzzer
void couperTir() {
  digitalWrite(pinLASER, ETEIND_LASER);
  digitalWrite(pinLED, ETEIND_LED);
  noTone(pinBUZZER);
  sonVerdictActif = false;
}

// Demarre le carillon du verdict
void lancerSonVerdict(unsigned long maintenant) {
  noteVerdict = 0;
  debutNoteVerdict = maintenant;
  sonVerdictActif = true;
  tone(pinBUZZER, verdictDrip ? NOTES_VALIDATION[0] : NOTES_REFUS[0]);
}

// Fait avancer le carillon note par note, sans bloquer la boucle
void actualiserSonVerdict(unsigned long maintenant) {
  if (!sonVerdictActif) return;
  unsigned long duree = verdictDrip ? DUREES_VALIDATION[noteVerdict]
                                    : DUREES_REFUS[noteVerdict];
  if (maintenant - debutNoteVerdict < duree) return;

  debutNoteVerdict = maintenant;
  noteVerdict++;
  int nombreNotes = verdictDrip ? NOMBRE_NOTES_VALIDATION : NOMBRE_NOTES_REFUS;
  if (noteVerdict >= nombreNotes) {
    noTone(pinBUZZER);
    sonVerdictActif = false;
  } else {
    tone(pinBUZZER, verdictDrip ? NOTES_VALIDATION[noteVerdict]
                                : NOTES_REFUS[noteVerdict]);
  }
}
```

```
// Change d'etat et force le rafraichissement de l'ecran
void changerEtat(EtatTir nouvelEtat) {
  etatTir = nouvelEtat;
  feuAffiche = false;
  dernierPourcentage = -1;
}

// Indicateur orange : le compteur est vide
void afficherChargeurVide() {
  analogWrite(pinLED_R, 255);
  analogWrite(pinLED_V, VERT_ORANGE);  // Rouge + un peu de vert = orange.
}

// Compteur vide : on coupe tout et on l'affiche
void signalerVide() {
  couperTir();
  afficherChargeurVide();
  changerEtat(VIDE);
}

// Mise en veille : ecran, retroeclairage et LED eteints
void entrerEnVeille() {
  couperTir();
  analogWrite(pinLED_R, 0);
  analogWrite(pinLED_V, 0);
  digitalWrite(TRIG, LOW);
  clicsCourts = 0;
  attendreRelachement = false;
  changerEtat(VEILLE);
  lcd.clear();
  lcd.noDisplay();
  lcd.noBacklight();
}

// Sortie de veille, retour a l'ecran d'accueil
void reveiller() {
  derniereActivite = millis();
  changerEtat(REPOS);
  clicsCourts = 0;
  attendreRelachement = true;
  systemeActive = false;
  if (munitions == 0) afficherChargeurVide();
  else analogWrite(pinLED_V, 255);
  lcd.display();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEME PRET");
  lcd.setCursor(0, 1);
  lcd.print("MUNITIONS ");
  lcd.print(munitions);
  lcd.print("/");
  lcd.print(CAPACITE);
}

// Demarre le mode rafale et consomme la premiere charge
void commencerRafale(unsigned long maintenant) {
  if (munitions == 0) {
    signalerVide();
    return;
  }
  changerEtat(TIR_AUTO);
  munitions--;
  tirCycleActif = true;
  debutPhase = maintenant;
  dernierCycleAuto = 0;
  dernierEffet = maintenant;
  digitalWrite(pinLASER, ALLUME_LASER);
  digitalWrite(pinLED, ALLUME_LED);
  analogWrite(pinLED_R, 255);
  analogWrite(pinLED_V, 0);
  tone(pinBUZZER, FREQUENCE_TIR_DEBUT);
}

// Coeur du programme : lit le bouton et fait avancer la machine a etats
void actualiserCharge() {
  int lecture = digitalRead(pinBTN);
  unsigned long maintenant = millis();
```

```
  actualiserSonVerdict(maintenant);

  if (verdictAffiche && maintenant - debutVerdict >= DUREE_VERDICT) {
    verdictAffiche = false;
    feuAffiche = false;
    dernierPourcentage = 101;
  }

  if (lecture != derniereLecture) {
    dernierChangement = maintenant;
    derniereLecture = lecture;
  }

  // En veille, seul un appui long reveille l'outil
  if (etatTir == VEILLE) {
    if (lecture == LOW && maintenant - dernierChangement >= DUREE_REVEIL) {
      reveiller();
    }
    return;
  }

  if (lecture == LOW || lecture != etatBouton ||
      etatTir != REPOS || clicsCourts > 0) {
    derniereActivite = maintenant;
  } else if (maintenant - derniereActivite >= DELAI_VEILLE_AUTO) {
    entrerEnVeille();
    return;
  }

  // Recharge en cours : degrade orange vers vert et bips qui montent
  if (etatTir == RECHARGE) {
    unsigned long ecoule = maintenant - debutPhase;
    if (ecoule >= DUREE_RECHARGE) {
      munitions = CAPACITE;
      couperTir();
      analogWrite(pinLED_R, 0);
      analogWrite(pinLED_V, 255);
      changerEtat(CONFIRMATION);
      debutPhase = maintenant;
      tone(pinBUZZER, 1400);
      attendreRelachement = true;
      clicsCourts = 0;
    } else if (maintenant - dernierEffet >= 20) {
      dernierEffet = maintenant;
      int progression = ecoule * 255UL / DUREE_RECHARGE;
      analogWrite(pinLED_R, 255 - progression);
      analogWrite(pinLED_V, VERT_ORANGE + progression * (255 - VERT_ORANGE) / 255);
      if (ecoule % 400 < 60) {
        unsigned int frequence = static_cast<unsigned int>(700 + ecoule / 2);
        tone(pinBUZZER, frequence);
      } else {
        noTone(pinBUZZER);
      }
    }
    return;
  }

  // Court message de confirmation a la fin de la recharge
  if (etatTir == CONFIRMATION) {
    unsigned long ecoule = maintenant - debutPhase;
    if (ecoule >= DUREE_BIP_PRET) {
      noTone(pinBUZZER);
    }
    if (ecoule >= DUREE_CONFIRMATION) {
      changerEtat(REPOS);
      dernierPourcentage = 101;
    }
    return;
  }

  // On attend le relachement du bouton avant d'accepter un nouveau geste
  if (attendreRelachement) {
    if (lecture == HIGH && maintenant - dernierChangement >= 30) {
      attendreRelachement = false;
      etatBouton = HIGH;
      feuAffiche = false;
      dernierPourcentage = 101;
```

```
    }
    return;
  }

  if (lecture == HIGH) {
    digitalWrite(pinLASER, ETEIND_LASER);
    digitalWrite(pinLED, ETEIND_LED);
  }

  if (maintenant - dernierChangement < ANTI_REBOND) return;

  // Un train de clics vient de se terminer : on choisit la fonction
  if (etatTir == REPOS && clicsCourts > 0 &&
      maintenant - dernierRelachement > DELAI_DOUBLE_CLIC &&
      (lecture == HIGH || dernierChangement - dernierRelachement > DELAI_DOUBLE_CLIC)) {
    int nombreClics = clicsCourts;
    clicsCourts = 0;
    feuAffiche = false;
    dernierPourcentage = 101;

    // Trois clics : recharge du compteur
    if (nombreClics == 3) {
      changerEtat(RECHARGE);
      debutPhase = maintenant;
      dernierEffet = maintenant - 20;
      afficherChargeurVide();
      return;
    }

    // Un ou deux clics : verdict de l'analyseur de style
    if (nombreClics == 1 || nombreClics == 2) {
      verdictDrip = nombreClics == 1;
      randomSeed(micros());
      indexVerdict = random(0, verdictDrip ? 4 : 3);
      debutVerdict = maintenant;
      verdictAffiche = true;
      lancerSonVerdict(maintenant);
    }
  }

  // Front detecte sur le bouton
  if (lecture != etatBouton) {
    etatBouton = lecture;
    if (lecture == HIGH) {
      bool clicCourt = etatTir == ATTENTE_CLIC &&
                       dernierChangement - debutAppui <= DUREE_CLIC_COURT;
      clicsCourts = clicCourt ? clicsCourts + 1 : 0;
      dernierRelachement = dernierChangement;
      couperTir();
      if (munitions == 0) afficherChargeurVide();
      else {
        analogWrite(pinLED_R, 0);
        analogWrite(pinLED_V, 255);
      }
      changerEtat(REPOS);
      dernierPourcentage = 101;
      if (clicsCourts == 4) entrerEnVeille();
      return;
    }

    if (dernierChangement - dernierRelachement > DELAI_DOUBLE_CLIC) {
      clicsCourts = 0;
    }
    verdictAffiche = false;
    if (sonVerdictActif) {
      noTone(pinBUZZER);
      sonVerdictActif = false;
    }
    changerEtat(ATTENTE_CLIC);
    systemeActive = true;
    debutAppui = dernierChangement;
    debutCharge = maintenant;
    dernierEffet = maintenant;
    debutPulsation = maintenant;
  }

  if (lecture == HIGH) return;
```

```
  // Appui long : rafale si un clic a precede, sinon charge simple
  if (etatTir == ATTENTE_CLIC) {
    if (maintenant - debutAppui > DUREE_CLIC_COURT) {
      if (clicsCourts >= 2) {
        clicsCourts = 0;
        couperTir();
        changerEtat(REPOS);
        dernierPourcentage = 101;
        attendreRelachement = true;
        return;
      }
      bool automatique = clicsCourts == 1;
      clicsCourts = 0;
      if (munitions == 0) {
        signalerVide();
      } else if (automatique) {
        commencerRafale(maintenant);
      } else {
        changerEtat(CHARGEMENT);
        debutCharge = maintenant;
        debutPulsation = maintenant;
        tone(pinBUZZER, FREQUENCE_CHARGE_DEBUT);
      }
    }
    return;
  }

  // Mode rafale : une impulsion laser par cycle
  if (etatTir == TIR_AUTO) {
    if (maintenant - dernierEffet >= 5) {
      dernierEffet = maintenant;
      unsigned long ecoule = maintenant - debutPhase;
      unsigned long cycle = ecoule / PERIODE_TIR_AUTO;
      unsigned long phase = ecoule % PERIODE_TIR_AUTO;
      if (munitions == 0 && (cycle != dernierCycleAuto || phase >= DUREE_TIR_AUTO)) {
        signalerVide();
        return;
      }
      if (cycle != dernierCycleAuto) {
        dernierCycleAuto = cycle;
        tirCycleActif = munitions > 0;
        if (tirCycleActif) {
          munitions--;
          feuAffiche = false;
        }
      }
      bool actif = tirCycleActif && phase < DUREE_TIR_AUTO;
// La rafale envoie des impulsions distinctes aux recepteurs des cibles.
      digitalWrite(pinLASER, actif ? ALLUME_LASER : ETEIND_LASER);
      digitalWrite(pinLED, actif ? ALLUME_LED : ETEIND_LED);
      if (munitions == 0) afficherChargeurVide();
      else {
        analogWrite(pinLED_R, actif ? 255 : 30);
        analogWrite(pinLED_V, 0);
      }
      if (actif) {
        float restant = 1.0f - static_cast<float>(phase) / DUREE_TIR_AUTO;
        unsigned int frequence = static_cast<unsigned int>(
            FREQUENCE_TIR_FIN + (FREQUENCE_TIR_DEBUT - FREQUENCE_TIR_FIN) * restant * restant);
        tone(pinBUZZER, frequence);
      } else {
        noTone(pinBUZZER);
      }
    }
    return;
  }

  // Courte pause entre la fin de la charge et le depart du tir
  if (etatTir == PAUSE_TIR) {
    if (maintenant - debutPhase >= DUREE_PAUSE) {
      if (munitions == 0) {
        signalerVide();
        return;
      }
      munitions--;
      changerEtat(TIR);
      debutPhase = maintenant;
      dernierEffet = maintenant;
```

```
      digitalWrite(pinLED, ALLUME_LED);
      digitalWrite(pinLASER, ALLUME_LASER);
      tone(pinBUZZER, FREQUENCE_TIR_DEBUT);
    }
    return;
  }

  // Tir simple en cours
  if (etatTir == TIR) {
    unsigned long ecoule = maintenant - debutPhase;
// Apres le son du tir, le laser reste allume jusqu'au relachement.
    digitalWrite(pinLASER, ALLUME_LASER);
    digitalWrite(pinLED, ALLUME_LED);
    if (ecoule >= DUREE_SON_TIR) {
      noTone(pinBUZZER);
    } else if (maintenant - dernierEffet >= 5) {
      dernierEffet = maintenant;
      float restant = 1.0f - static_cast<float>(ecoule) / DUREE_SON_TIR;
      unsigned int frequence = static_cast<unsigned int>(
          FREQUENCE_TIR_FIN + (FREQUENCE_TIR_DEBUT - FREQUENCE_TIR_FIN) * restant * restant);
      tone(pinBUZZER, frequence);
    }
    return;
  }

  // A partir d'ici, seule la charge nous interesse
  if (etatTir != CHARGEMENT) return;

  // Charge : degrade vert vers rouge, clignotement et son qui monte
  unsigned long ecoule = maintenant - debutCharge;
  if (ecoule >= DUREE_CHARGE) {
    noTone(pinBUZZER);
    analogWrite(pinLED_R, 255);
    analogWrite(pinLED_V, 0);
    etatTir = PAUSE_TIR;
    debutPhase = maintenant;
    dernierPourcentage = -1;
  } else if (maintenant - dernierEffet >= 10) {
    dernierEffet = maintenant;
    int progression = ecoule * 255UL / DUREE_CHARGE;
    unsigned long periode = 360 - ecoule * 290UL / DUREE_CHARGE;
    if (maintenant - debutPulsation >= periode) {
      debutPulsation = maintenant;
    }
    bool impulsion = maintenant - debutPulsation < periode / 2;
    int luminosite = impulsion ? 255 : 45;
    analogWrite(pinLED_R, progression * luminosite / 255);
    analogWrite(pinLED_V, (255 - progression) * luminosite / 255);
    float avancement = static_cast<float>(ecoule) / DUREE_CHARGE;
    unsigned int frequence = static_cast<unsigned int>(
        FREQUENCE_CHARGE_DEBUT +
        (FREQUENCE_CHARGE_FIN - FREQUENCE_CHARGE_DEBUT) * avancement * avancement);
    tone(pinBUZZER, frequence);
  }
}

// Fonction telemetre : impulsion sur TRIG puis largeur de l'impulsion ECHO
float mesurerDistance() {

  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG, LOW);

  unsigned long duree = pulseIn(ECHO, HIGH, 30000);   // 30 ms maximum, au dela on considere qu'il n'y a pas
d'echo
  actualiserCharge();

  if (duree == 0) {
    return -1;
  }

  return duree * 0.0343 / 2;   // Son a 343 m/s, aller-retour divise par deux
}
```

```
// Barre de progression, utilisee pour la charge et pour la recharge
void afficherChargement() {
  unsigned long maintenant = millis();
  if (dernierPourcentage >= 0 && maintenant - dernierAffichageCharge < 100) {
    return;
  }
  dernierAffichageCharge = maintenant;

  bool recharge = etatTir == RECHARGE;
  unsigned long duree = recharge ? DUREE_RECHARGE : DUREE_CHARGE;
  unsigned long ecoule = maintenant - (recharge ? debutPhase : debutCharge);
  int pourcentage = ecoule >= duree ? 100 : ecoule * 100UL / duree;
  if (pourcentage == dernierPourcentage) {
    return;
  }

  if (dernierPourcentage < 0) {
    lcd.setCursor(0, 0);
    lcd.print(recharge ? "RECHARGE        " : "CHARGEMENT      ");
  }
  dernierPourcentage = pourcentage;

  lcd.setCursor(0, 1);
  lcd.print("[");
  for (int i = 0; i < 10; i++) {
    lcd.print(i < pourcentage / 10 ? "#" : " ");
  }
  lcd.print("]");
  if (pourcentage < 100) lcd.print(" ");
  if (pourcentage < 10) lcd.print(" ");
  lcd.print(pourcentage);
  lcd.print("%");
}

// Ecran de repos : distance mesuree et charge restante
void afficherDistance() {

  float distance = mesurerDistance();
  if (etatTir != REPOS || munitions == 0 || clicsCourts > 0) {
    return;
  }
  dernierPourcentage = -1;

  lcd.setCursor(0, 0);
  lcd.print("DISTANCE   ");
  if (munitions < 10) lcd.print(" ");
  lcd.print(munitions);
  lcd.print("/20");
  lcd.setCursor(0, 1);
  lcd.print("                ");

  if (distance >= 0) {

    lcd.setCursor(0, 1);
    lcd.print(distance, 1);
    lcd.print(" cm");

  } else {

    lcd.setCursor(0, 1);
    lcd.print("AUCUNE MESURE");
  }

}

// Boucle principale : choisit uniquement ce qui doit s'afficher
void loop() {
  actualiserCharge();

  if (etatTir == VEILLE) {
    delay(5);
    return;
  }

  if (etatTir == CHARGEMENT || etatTir == PAUSE_TIR || etatTir == RECHARGE) {
    afficherChargement();
  } else if (etatTir == CONFIRMATION) {
    if (!feuAffiche) {
```

```
      lcd.setCursor(0, 0);
      lcd.print("PRET ! ");
      lcd.print(munitions);
      lcd.print("/");
      lcd.print(CAPACITE);
      lcd.print("    ");
      lcd.setCursor(0, 1);
      lcd.print("RECHARGE FINIE  ");
      feuAffiche = true;
    }
  } else if (etatTir == TIR || etatTir == TIR_AUTO) {
    if (!feuAffiche) {
      lcd.setCursor(0, 0);
      lcd.print(etatTir == TIR_AUTO ? "  MODE AUTO     " : "  >>> FEU ! <<< ");
      lcd.setCursor(0, 1);
      lcd.print("MUNITIONS ");
      if (munitions < 10) lcd.print(" ");
      lcd.print(munitions);
      lcd.print("/20 ");
      feuAffiche = true;
    }
  } else if (etatTir == REPOS && attendreRelachement && digitalRead(pinBTN) == LOW) {
    if (!feuAffiche) {
      lcd.setCursor(0, 0);
      lcd.print("RELACHEZ BOUTON ");
      lcd.setCursor(0, 1);
      lcd.print("POUR CONTINUER  ");
      feuAffiche = true;
    }
  } else if (etatTir == REPOS && verdictAffiche) {
    if (!feuAffiche) {
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print(verdictDrip ? "VERDICT: DRIP!" : "VERDICT PAS DRIP");
      lcd.setCursor(0, 1);
      lcd.print(verdictDrip ? MESSAGES_DRIP[indexVerdict]
                            : MESSAGES_PAS_DRIP[indexVerdict]);
      feuAffiche = true;
    }
  } else if (etatTir == ATTENTE_CLIC || (etatTir == REPOS && clicsCourts > 0)) {
    if (!feuAffiche) {
      lcd.setCursor(0, 0);
      lcd.print("CLICS : ");
      lcd.print(clicsCourts);
      lcd.print("       ");
      lcd.setCursor(0, 1);
      if (clicsCourts == 0) lcd.print("MAINTENIR:CHARGE");
      else if (clicsCourts == 1) lcd.print("MAINTENIR: AUTO ");
      else if (clicsCourts == 2) lcd.print("3 CLICS:RECHARGE");
      else lcd.print("4 CLICS: VEILLE ");
      feuAffiche = true;
    }
  } else if (etatTir == VIDE ||
             (etatTir == REPOS && munitions == 0)) {
    if (!feuAffiche) {
      lcd.setCursor(0, 0);
      lcd.print(munitions == 0 ? "CHARGEUR VIDE   " : "MAINTENIR: AUTO ");
      lcd.setCursor(0, 1);
      lcd.print("3 CLICS:RECHARGE");
      feuAffiche = true;
    }
  } else if (systemeActive && digitalRead(pinBTN) == HIGH &&
             (dernierPourcentage >= 0 || millis() - derniereMesure >= 250)) {
    afficherDistance();
    derniereMesure = millis();
  }

  actualiserCharge();
  delay(1);
}
```

Annexe B. Programme des cibles

Programme téléversé sur la seconde carte NodeMCU, celle qui gère les deux extraterrestres.

```
// Capteurs de lumiere loges dans le ventre des deux cibles
#define CAPTEUR_1 5  // D1
#define CAPTEUR_2 4  // D2

// LED RGB dans la tete de la cible 1
#define RGB1_G 14  // D5
#define RGB1_R 12  // D6
#define RGB1_B 13  // D7

// LED RGB dans la tete de la cible 2
#define RGB2_G 16  // D0
#define RGB2_R 0  // D3
#define RGB2_B 2  // D4

// Buzzer partage par les deux cibles
#define BUZZER 15  // D8

// Etat de chaque personnage : touche, et arme pour pouvoir l'etre
bool mort1 = false;
bool mort2 = false;
bool pret1 = false;
bool pret2 = false;
unsigned long touche1A = 0;
unsigned long touche2A = 0;

const unsigned long DELAI_REINITIALISATION = 5000;   // Retour automatique au vert apres l'impact

// Sequence de KO facon borne d'arcade : douze notes qui descendent
// Une frequence de 0 correspond a une courte pause.
const int NOTES_KO[] = {1500, 1050, 700, 0, 560, 750, 500, 680, 420, 320, 240, 180};
const unsigned long DUREES_KO[] = {45, 55, 80, 35, 90, 75, 90, 75, 110, 140, 180, 240};
const int NOMBRE_NOTES = sizeof(NOTES_KO) / sizeof(NOTES_KO[0]);
const int NIVEAU_LASER = LOW;  // Le recepteur passe a LOW quand il est eclaire
const unsigned long INTERVALLE_AFFICHAGE = 200;
unsigned long dernierAffichage = 0;
unsigned long debutNote = 0;
int noteActuelle = 0;
bool buzzerActif = false;

// Personnage en vie
void vertRGB(int G, int R, int B) {
  digitalWrite(G, HIGH);
  digitalWrite(R, LOW);
  digitalWrite(B, LOW);
}

// Personnage touche
void rougeRGB(int G, int R, int B) {
  digitalWrite(G, LOW);
  digitalWrite(R, HIGH);
  digitalWrite(B, LOW);
}

// Lance la sequence de KO
void sonner(unsigned long maintenant) {
  noteActuelle = 0;
  debutNote = maintenant;
  buzzerActif = true;
  tone(BUZZER, NOTES_KO[noteActuelle]);
}

// Fait avancer la sequence note par note, sans bloquer la boucle
void mettreAJourSon(unsigned long maintenant) {
  if (!buzzerActif || maintenant - debutNote < DUREES_KO[noteActuelle]) {
    return;
  }

  debutNote = maintenant;
  noteActuelle++;
  if (noteActuelle >= NOMBRE_NOTES) {
    noTone(BUZZER);
    buzzerActif = false;
  } else if (NOTES_KO[noteActuelle] == 0) {
```

```
    noTone(BUZZER);
  } else {
    tone(BUZZER, NOTES_KO[noteActuelle]);
  }
}

// Configuration des broches, les deux personnages demarrent en vie
void setup() {

  Serial.begin(115200);

  pinMode(CAPTEUR_1, INPUT);
  pinMode(CAPTEUR_2, INPUT);

  pinMode(RGB1_G, OUTPUT);
  pinMode(RGB1_R, OUTPUT);
  pinMode(RGB1_B, OUTPUT);

  pinMode(RGB2_G, OUTPUT);
  pinMode(RGB2_R, OUTPUT);
  pinMode(RGB2_B, OUTPUT);

  pinMode(BUZZER, OUTPUT);
  digitalWrite(BUZZER, LOW);

  vertRGB(RGB1_G, RGB1_R, RGB1_B);
  vertRGB(RGB2_G, RGB2_R, RGB2_B);

  Serial.println("================================");
  Serial.println("  SYSTEME DES PERSONNAGES");
  Serial.println("================================");
  Serial.println("Personnage 1 : VIVANT");
  Serial.println("Personnage 2 : VIVANT");
  Serial.println("Attente d'un etat sans laser sur chaque capteur");
}

// Lecture des deux capteurs et mise a jour de chaque personnage
void loop() {

  int capteur1 = digitalRead(CAPTEUR_1);
  int capteur2 = digitalRead(CAPTEUR_2);
  unsigned long maintenant = millis();

  mettreAJourSon(maintenant);

  // Trace periodique de l'etat des capteurs sur le port serie
  if (maintenant - dernierAffichage >= INTERVALLE_AFFICHAGE) {
    dernierAffichage = maintenant;
    Serial.print("Capteur 1 = ");
    Serial.print(capteur1);
    Serial.print(" | Capteur 2 = ");
    Serial.println(capteur2);
  }

  // Le personnage 1 etait touche : il revient au vert apres le delai
  if (mort1 && maintenant - touche1A >= DELAI_REINITIALISATION) {
    mort1 = false;
    pret1 = false;
    vertRGB(RGB1_G, RGB1_R, RGB1_B);
    Serial.println("Personnage 1 : REINITIALISE");
  }

  // Pas de faisceau sur le capteur : le personnage 1 est arme
  if (!mort1 && capteur1 != NIVEAU_LASER) {
    pret1 = true;
  }

  // Le faisceau touche un personnage arme : il est mis hors jeu
  if (!mort1 && pret1 && capteur1 == NIVEAU_LASER) {

    mort1 = true;
    pret1 = false;
    touche1A = maintenant;

    rougeRGB(RGB1_G, RGB1_R, RGB1_B);
    sonner(maintenant);

    Serial.println("💥 PERSONNAGE 1 TOUCHE !");
```

```
  }

  // Meme logique, pour le personnage 2
  if (mort2 && maintenant - touche2A >= DELAI_REINITIALISATION) {
    mort2 = false;
    pret2 = false;
    vertRGB(RGB2_G, RGB2_R, RGB2_B);
    Serial.println("Personnage 2 : REINITIALISE");
  }

  if (!mort2 && capteur2 != NIVEAU_LASER) {
    pret2 = true;
  }

  if (!mort2 && pret2 && capteur2 == NIVEAU_LASER) {

    mort2 = true;
    pret2 = false;
    touche2A = maintenant;

    rougeRGB(RGB2_G, RGB2_R, RGB2_B);
    sonner(maintenant);

    Serial.println("💥 PERSONNAGE 2 TOUCHE !");
  }

  delay(5);
}
```

Annexe C. Schéma de branchement des composants

Ce schéma est représentatif du montage. Il montre la répartition des connexions entre les deux cartes NodeMCU ESP8266, avec le repère sérigraphié de chaque broche et le numéro de GPIO correspondant. Les affectations reprises ici sont celles déclarées en tête des deux programmes, en annexe A et en annexe B.

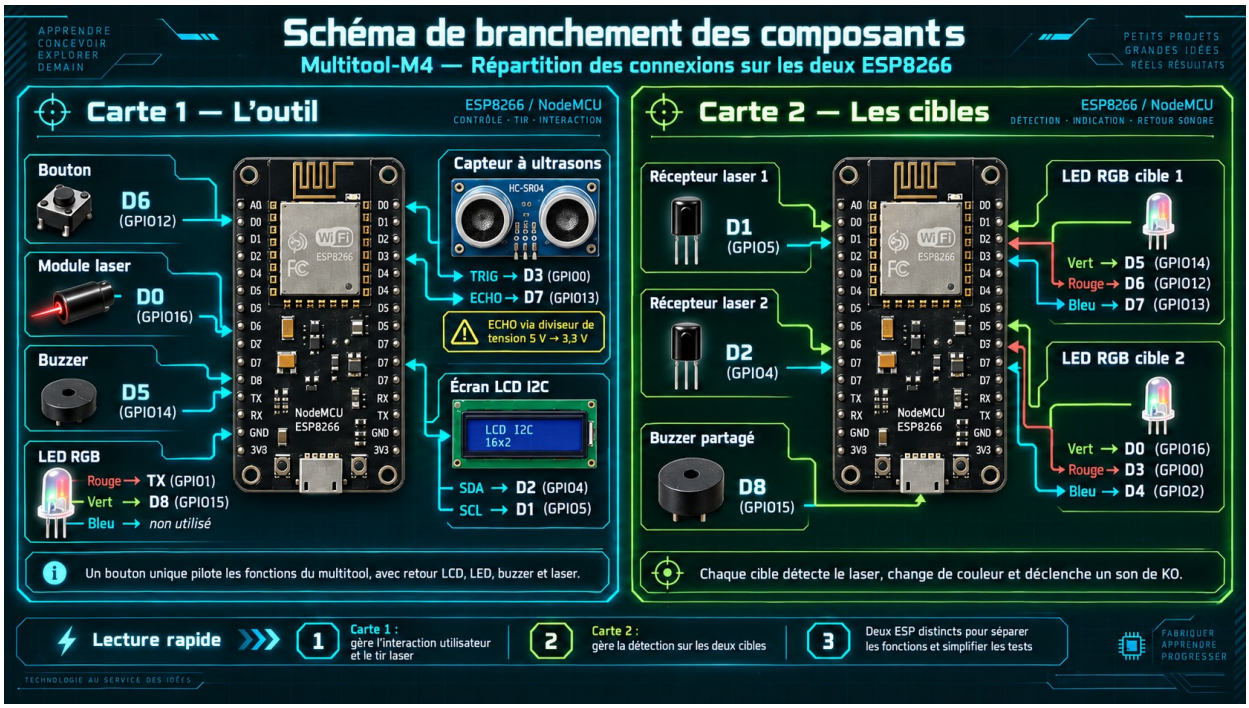


Figure 7 : schéma de branchement des composants sur les deux cartes.